

Solution Requirements

Phase 2 — Requirement Analysis

This phase turns the idea from Phase 1 into clear, checkable requirements. It describes who uses the system, what the system must do, how well it must do it, and what it is built with.

1. Stakeholders and Users

- **End user** — a registered person who creates budget plans.
- **Guest visitor** — a person who can only see the landing, register and login pages.
- **Development team** — builds and maintains the application.
- **External services** — Google Gemini (AI) and shopping websites (link destinations only).

2. User Stories

- As a new visitor, I want to create an account so my plans are saved under my name.
- As a user, I want to log in and log out safely so nobody else can see my plans.
- As a homeowner, I want to enter my budget and needs so I get a suggested split across lighting, fans, furniture and other items.
- As a party host, I want to enter guests, party type and needs so I get a cost plan for catering, decoration and entertainment.
- As a jewelry buyer, I want to upload a photo of my outfit so the suggestions match its colours and style.
- As a shopper, I want a link for each suggested item so I can go straight to the shop.
- As a returning user, I want to see my earlier recommendations so I can compare or reuse them.

3. Functional Requirements

Account management

- FR-1: The system shall let a visitor register with a username, an email and a password.
- FR-2: The system shall reject registration if the username already exists.
- FR-3: The system shall store passwords only in hashed form.
- FR-4: The system shall let a registered user log in with username and password.
- FR-5: The system shall reject wrong credentials with an error.
- FR-6: The system shall let a logged-in user log out, after which the old session must no longer work.

Access control

- FR-7: Dashboard, planners and history pages shall be available only to logged-in users.
- FR-8: A session shall expire automatically after 30 minutes.

Home planner

- FR-9: The user shall enter total budget, number of lights, fans, furniture pieces and dining tables.
- FR-10: The user shall select which rooms apply: living room, kitchen, bedroom.

- FR-11: The user may add free-text additional requirements.

Party planner

- FR-12: The user shall enter total budget, number of guests and party type.
- FR-13: The user may enter a venue type and choose whether catering, decoration and entertainment are needed.
- FR-14: The user may add free-text additional requirements.

Jewelry planner

- FR-15: The user shall enter total budget and occasion.
- FR-16: The user may enter style preferences.
- FR-17: The user may upload an outfit image, which is saved on the server and sent to the AI for analysis.

AI recommendations

- FR-18: The system shall send a structured prompt to Gemini and receive a JSON answer.
- FR-19: The result shall contain a category-wise breakdown with item name, description, estimated price and quantity, plus remaining budget and extra suggestions (for jewelry: outfit analysis, recommendations and styling tips).
- FR-20: The system shall tolerate answers wrapped in markdown code fences.

Shopping links

- FR-21: The system shall create search links for every item from its search terms.
- FR-22: Home links go to Amazon India, Flipkart, IKEA India and Myntra.
- FR-23: Party links go to Amazon India, Flipkart, Swiggy, Zomato and BookMyShow.
- FR-24: Jewelry links go to Amazon India, Flipkart, Tanishq and CaratLane.

Dashboard and history

- FR-25: Every generated recommendation shall be saved with an ID, timestamp, type, input and result.
- FR-26: The dashboard shall show the latest five recommendations of the logged-in user.
- FR-27: The history page shall show all recommendations of the logged-in user, newest first.

4. Non-Functional Requirements

Security

- Passwords hashed with bcrypt.
- Sessions use JWT (HS256) in an HTTP-only cookie so scripts in the browser cannot read it.
- Logged-out tokens are blacklisted.
- Secrets (API key, signing key) come from environment variables, never from source code.

Usability

- One simple form per planner, clear labels, results shown on the same page.
- Works in a normal desktop or mobile browser without installation.

Performance

- Pages other than AI calls should load almost immediately; AI calls depend on Gemini response time, typically a few seconds.

Reliability

- If the AI wraps or pads its JSON, the parser should still recover the data.
- Invalid form data should be rejected with a clear error instead of crashing.

Maintainability

- Routes, authentication, AI logic and data models live in separate files.
- Adding a new shopping platform means adding one line to a dictionary.

Portability

- Runs on Windows, macOS and Linux with Python and pip.

5. Technology Stack and Justification

- **Python** — readable, strong AI ecosystem, quick to learn.
- **FastAPI** — modern, fast web framework with built-in input validation and automatic API documentation.
- **Uvicorn** — server used to run FastAPI.
- **Jinja2 templates with HTML/CSS/JavaScript** — simple page rendering without a heavy front-end framework.
- **Google Gemini (google-generativeai)** — provides text and image understanding for recommendations.
- **python-jose** — creates and verifies JWT tokens.
- **Passlib with bcrypt** — safe password hashing.
- **Pillow** — opens the uploaded outfit image before sending it to Gemini.
- **python-multipart** — lets FastAPI read form and file uploads.
- **python-dotenv** — loads settings from a .env file.

6. Data Requirements

RegisterUser: username, email, password.

UserInDB: username, email, hashed_password.

HomeBudgetInput: total_budget, num_lights, num_fans, num_furniture, num_dining_tables, has_living_room, has_kitchen, has_bedroom, additional_requirements.

PartyBudgetInput: total_budget, num_guests, party_type, venue_type, needs_catering, needs_decoration, needs_entertainment, additional_requirements.

JewelryBudgetInput: total_budget, occasion, preferences.

History record: id, timestamp, type (home, party or jewelry), input, result.

7. Environment and Configuration Requirements

- **GOOGLE_API_KEY** — credential for the Gemini API. The application refuses to start without it.
- **SECRET_KEY** — secret used to sign login tokens. If missing, a weak default is used, so it must always be set.
- A .env file created from .env.example, kept out of GitHub.
- Internet access at run time.

8. Hardware and Software Requirements

- Any computer that can run Python 3 (a modern laptop is enough).
- A modern web browser (Chrome, Edge, Firefox, Safari).
- pip and, recommended, a virtual environment.
- A code editor such as VS Code.

9. Assumptions

- Users are in India and think in INR.
- AI prices are estimates and may differ from actual shop prices.
- One server process is used, so in-memory storage is acceptable for the prototype.

10. Constraints

- Users, tokens and history are lost when the server restarts.
- Depends on the availability, limits and cost of the Gemini API.
- Shopping links are search links, not live product listings.

11. Out of Scope for This Version

Payments, ordering, live price tracking, email verification, password reset, multi-user shared plans, and a permanent database.